INFORMATIONSBLATT

IB02 – Angriffe verstehen: SQL-Injection und XSS

Dynamische Webseiten mit PHP · LS09 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie erkennen, wie SQL-Injection und gespeichertes Cross-Site-Scripting funktionieren, und wenden die zwei Regeln an, die beide Angriffe zuverlässig verhindern.

1 | Eine unsichere Suche

Diese Suche setzt die Eingabe direkt in den SQL-Befehl ein. Mit einem normalen Namen funktioniert sie tadellos:

suche_unsicher.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Unsichere Suche</title></head>
<body>

<?php
  require_once "db_verbindung.php";

  $name = $_GET["name"];
  $sql  = "SELECT name, datum, tel FROM reservierungen WHERE name = '$name'";   // GEFÄHRLICH!
  $ergebnis = $conn->query($sql);

  echo "<p>Befehl: " . htmlspecialchars($sql) . "</p>";
  while ($r = $ergebnis->fetch_assoc()) {
    echo "<p>" . htmlspecialchars($r["name"] . " · " . $r["datum"] . " · " . $r["tel"]) . "</p>";
  }
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS



Abbildung 1: Mit einer normalen Eingabe: genau ein Treffer.

Jetzt gibt jemand statt eines Namens `x' OR '1'='1` ein:

SO SIEHT ES IM BROWSER AUS



Abbildung 2: Die Eingabe hat den Befehl verändert: Alle Namen und Telefonnummern werden ausgegeben.

Das Hochkomma in der Eingabe beendet den Text im SQL-Befehl, und `OR '1'='1'` hängt eine Bedingung an, die immer wahr ist. So kann ein Angreifer nicht nur lesen: Mit anderen Eingaben lassen sich je nach Programmierung auch Anmeldungen umgehen oder Daten verändern. Dieser Angriff heißt **SQL-Injection** und ist seit Jahren eine der häufigsten Ursachen für Datenlecks.

2 | Dieselbe Suche – sicher

suche_sicher.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Sichere Suche</title></head>
<body>

<?php
  require_once "db_verbindung.php";

  $name = $_GET["name"];
  $stmt = $conn->prepare("SELECT name, datum, tel FROM reservierungen WHERE name = ?");
  $stmt->bind_param("s", $name);
  $stmt->execute();
  $ergebnis = $stmt->get_result();

  echo "<p>Treffer: " . $ergebnis->num_rows . "</p>";
  while ($r = $ergebnis->fetch_assoc()) {
    echo "<p>" . htmlspecialchars($r["name"] . " · " . $r["datum"] . " · " . $r["tel"]) . "</p>";
  }
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

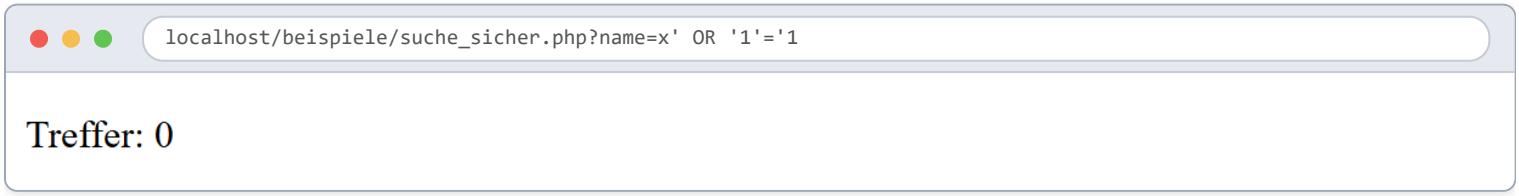


Abbildung 3: Mit Prepared Statement: Die Eingabe wird als Name gesucht – kein Treffer.

📌 **REGEL 1 – SQL**

Werte aus Formularen, Adressen (GET) oder Cookies gelangen **nur über Platzhalter** in SQL-Befehle. Niemals Variablen in den SQL-Text einsetzen. Das gilt auch für scheinbar harmlose Felder und für Daten, die „nur intern“ genutzt werden.

3 | Gespeichertes Cross-Site-Scripting

In LS03 wurde eingeschleustes HTML nur dem Angreifer selbst angezeigt. Jetzt werden Eingaben **gespeichert** und anderen gezeigt – etwa dem Personal in der Reservierungsliste. Ein eingeschleustes Skript läuft dann im Browser von Frau Ricci.

Ausgabe gespeicherter Daten beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Gespeichertes XSS</title></head>
<body>

<?php
  // So stünde ein Datensatz in der Datenbank, den jemand über das Formular eingeschleust hat:
  $r = ["name" => "<b style='color:red;font-size:28px'>Alle Reservierungen storniert!</b>"];

  echo "<p>Ohne Schutz: " . $r["name"] . "</p>";
  echo "<p>Mit Schutz: " . htmlspecialchars($r["name"]) . "</p>";
?>

</body>
</html>
```

👁 **SO SIEHT ES IM BROWSER AUS**



Abbildung 4: Oben wird das gespeicherte HTML ausgeführt, unten harmlos angezeigt.

📌 **REGEL 2 – HTML**

Alles, was aus einer Eingabe oder aus der Datenbank stammt, wird bei der Ausgabe mit `htmlspecialchars()` behandelt. Prepared Statements schützen nur die Datenbank – nicht den Browser!

4 | Der Weg einer Eingabe

Station	Schutz	gelernt in
Formular → PHP	prüfen: vorhanden, sinnvoller Bereich, erlaubter Wert (Whitelist)	LS04
PHP → Datenbank	Prepared Statement mit `?` und `bind_param()`	LS09
Datenbank / PHP → Browser	`htmlspecialchars()`	LS03, LS09